

תרגיל בית 7

ALGO + OOP

הנחיות כלליות :

- קראו **היטב** את השאלות והקפידו שהתכניות שלכם פועלות בהתאם לנדרש.
- את התרגיל יש לפתור לבד!
- הקפידו על כללי ההגשה המפורסמים באתר. בפרט, יש להגיש את כל השאלות יחד בקובץ `ex7_012345678.py` המצורף לתרגיל, לאחר החלפת הספרות 012345678 במספר ת.ז שלכם, כל 9 הספרות כולל ספרת הביקורת.
- מועד אחרון להגשה : כמפורסם באתר.
- בדיקה עצמית: כדי לוודא את נכונותן ואת עמידותן של התוכניות לקלטים שגויים, בכל שאלה, הריצו את תוכניתכם עם מגוון קלטים שונים, אלה שהופיעו כדוגמאות בתרגיל וקלטים נוספים עליהם חשבתם (וודאו כי הפלט נכון וכי התוכנית אינה קורסת).
- היות ובדיקת התרגילים עשויה להיות אוטומטית, יש להקפיד על פלטים מדויקים על פי הדוגמאות (עד לרמת הרווח).
- אופן ביצוע התרגיל: בתרגיל זה עליכם להשלים את הקוד בקובץ המצורף.
- אין לשנות את שמות המשתנים שכבר מופיעים בקובץ השלד של התרגיל.
- **יש לעבוד עם המשתנים שמופיעים בשלד התרגיל**. על הקוד של כל שאלה לעבוד ולספק את התוצאה הדרושה עבור קלט שיוזן במשתנים שמופיעים בשלד (המשתנים שלידם סימני שאלה ומחכים לקלט כפי שראינו בדוגמא מהתרגול). יחד עם זאת, אתם רשאים להוסיף משתנים נוספים כראותם עינכם.
- אין למחוק את ההערות שמופיעות בשלד.
- את התרגיל האחרון יש להגיש כקובץ PDF `ex7_012345678.pdf` כאשר את המספרים יש להחליף בת.ז. ההגשה היא בתיבה נפרדת במודל "הגשת פתרון PDF".

בכל השאלות ניתן להניח את תקינות הקלט על פי המפורט בשאלה

בתרגיל זה נממש חלק מתוכנה לניהול מוזיאון. המערכת תאפשר ניהול נוח של מיצגי האומנות המוצגים במוזיאון ושל המנויים שלו.

- שאלות המסומנות בכוכבית דורשות ידע אשר ילמד בשיעור OOP השני.
- בשאלות שניתן בהן חופש פעולה לגבי צורת המימוש של המחלקות – כל פתרון שעובד על פי הדרישות יתקבל.
- ניתן להוסיף מתודות מחלקה כרצונכם.
- מומלץ בחום לקרוא תחילה את כל התרגיל ולראות כיצד ניתן לתכנן את הפונקציות בהתאם.

שאלה 1

בשאלה זו נממש את המחלקה **ArtDisplay** אשר מייצגת מיצג אומנות. כל מיצג (אובייקט מסוג ArtDisplay) מכיל את השדות הבאים:

1. **name** - שדה מסוג מחרוזת (string) המציין את שם המיצג.
2. **date** - שדה מסוג מחרוזת (string) המציין את תאריך היצירה של המיצג (מחרוזת מהצורה DD.MM.YYYY למשל "22.1.1990" או "21.12.1888").
3. **art_type** – שדה מסוג מחרוזת המציין את סוג המיצג (ערך המחרוזת יהיה "statue" או "painting" בלבד).
4. **preserving_date** - שדה מסוג מחרוזת המציין את תאריך חידוש המיצג (מחרוזת מהצורה DD.MM.YYYY כמתואר לעיל).
5. **worth** – שדה מסוג int המציין את שווייה הכספי של המיצג במליוני דולרים.

א. ממשו את בנאי המחלקה:

`__init__(self, name, date, art_type, preserving_date, worth)`

הבנאי מקבל את הנתונים הדרושים ושומר אותם בשדות הרלוונטיים של האובייקט החדש שהוא מייצר עבורו.

ניתן להניח שהקלט שנשלח לבנאי תקין לחלוטין מלבד הפרמטר worth עבורו יש לוודא שהוא גדול מאפס. אם ערכו קטן או שווה לאפס, על הבנאי להעלות ValueError עם הודעת השגיאה "Invalid worth value".

ב. עבור כל מיצג אומנות ניתן לבצע פעולת שימור תקופתית. לאחר ביצוע פעולת השימור יש לעדכן את ערך השדה המציין את תאריך שימור המיצג. הוסיפו למחלקה ArtDisplay את המתודה **`change_preserving_date(self, new_date)`** שמשנה את השדה המציין את תאריך שימור המיצג לתאריך ששמור במחרוזת new_date.

שאלה 2

בשאלה זו נממש את המחלקה **MuseumSubscriber** אשר מייצגת מנוי למוזיאון. כל מנוי (אובייקט

מסוג MuseumSubscriber) מכיל את השדות הבאים:

1. **name** – שדה מסוג מחרוזת המציין את שם המנוי.
 2. **entries_left** – שדה המציין כמה כניסות נותרו למנוי. שדה עשוי להיות מטיפוס מספר (int) – ואז ערכו מציין כמה כניסות נותרו למנוי, או מטיפוס מחרוזת – ואז הדבר מעיד על כך שמדובר במנוי שנתי וערכו של השדה מציין את תאריך תום המנוי.
 3. **favorites** – שדה מסוג רשימה המכילה אובייקטים המייצגים מיצגי אומנות האהובות על המנוי (אובייקטים מסוג ArtDisplay). ניתן להניח שברשימה זו אין חזרות.
- א. ממשו את בנאי המחלקה:

__init__(self, name, ticket_type, favorites)

הבנאי מקבל את הנתונים הדרושים כארגומנטים ומעדכן על פיהם את השדות הרלוונטיים באובייקט החדש שהוא מייצר. ניתן להניח שהקלט **תקין** לחלוטין.

הארגומנטים name ו-favorites שנשלחים לבנאי הינם מסוג מחרוזת ורשימה בהתאמה, ויש לשמור את ערכם לשדה בשם המתאים.

עם זאת, הארגומנט ticket_type שנשלח לבנאי יהיה תמיד מסוג מחרוזת, ויש לתרגם את ערכו לערך השדה entries_left על פי הפירוט הבא:

- אם ערך הארגומנט ticket_type הוא "1" אז יש לשמור את המספר 1 כ-int בשדה entries_left (מייצג מנוי שזכאי לכניסה בודדת).
- אם ערך הארגומנט ticket_type הוא "5" אז יש לשמור את המספר 5 כ-int בשדה entries_left (מייצג מנוי שזכאי ל-5 כניסות).
- אחרת, נניח שהמחרוזת הינה מהצורה DD.MM.YYYY ושהיא מציינת את תאריך תום המנוי השנתי (מנוי זה מאפשר כניסות ללא הגבלה).

דוגמאות לבניית אובייקטים:

```
>>>subs1 = MuseumSubscriber("John","1",[display1 ,display2])
>>>subs2 = MuseumSubscriber("Dany","5",[display1 ,display2])
>>>subs3 = MuseumSubscriber("Oren","12.1.2020",[display1 ,display2])
```

כאשר display1 ו display2 הם אובייקטים מסוג ArtDisplay.

ב. הוסיפו למחלקה MuseumSubscriber מימוש למתודה **set_entry** שמעדכנת כניסה למנויים.

- אם סוג המנוי הוא מנוי שנתי (עד תאריך מסוים), תודפס למסך ההודעה הבאה:
Welcome subscriber!
- אם לא מדובר במנוי שנתי, ולא נותרו למנוי כניסות (ערך השדה entries_left הוא אפס), אז תופיע ההודעה הבאה:
Please renew your subscription
- אם לא מדובר במנוי שנתי, ונותרו למנוי כניסות (ערך השדה entries_left גדול מאפס), יש להפחית כניסה מערך השדה ולהדפיס למסך:
Welcome! <entries_left> entries left
כאשר <entries_left> מציין כמה כניסות נשארו למנוי לאחר כניסה זו.

לדוגמא:

```
>>> subs2 = MuseumSubscriber("Hannah", "5", [display1 ,display2])
>>> subs2.set_entry()
Welcome! 4 entries left
>>> subs1 = MuseumSubscriber("Jhon", "1", [display1 ,display2])
>>> subs1.set_entry()
Welcome! 0 entries left
>>> subs1.set_entry()
Please renew your subscription
>>> subs3 = MuseumSubscriber("Dean", "11.12.2020", [display1 ,display2])
>>>subs3.set_entry()
Welcome subscriber!
```

ד. ממשו את המתודה **get_favorites()** שמחזירה את רשימת המיצגים המועדפים של המנוי (יש להחזיר את ערך השדה favorites שהינו רשימה של אובייקטים מסוג ArtDisplay).

שאלה 3

בשאלה זו נממש את המחלקה **Museum**, אשר תייצג מוזיאון ובו מיצגי אומנות ומנויים מסוגים שונים. למחלקה זו השדות הבאים:

1. subscribers – רשימה של מנויים (אובייקטים מסוג MuseumSubscriber).
2. art_displays – רשימה של מיצגי אומנות (אובייקטים מסוג ArtDisplay).

א. ממשו את בנאי המחלקה

__init__(self, art_displays)

אשר מקבל רשימה של מיצגי אומנות (רשימה של אובייקטים מסוג ArtDisplay). הבנאי ישמור בשדה art_displays עותק של ארגומנט זה. הבנאי גם ישמור רשימה ריקה בשדה subscribers.

ב. הוסיפו למחלקה Museum את המתודות הבאות:

- מתודה שתחזיר את רשימת (list) מיצגי האומנות (אובייקטים מסוג ArtDisplay).

get_art_displays(self)

- מתודה שתחזיר מיצג אומנות ספציפי (אובייקט ArtDisplay) לפי מחרוזת name.

get_art_display(self, name)

- מתודה שתוסיף מיצג אומנות לרשימת מיצגי האומנות במוזיאון. המתודה תקבל אובייקט מסוג ArtDisplay ותוסיף אותו לרשימת מיצגי האומנות במוזיאון.

add_art_display(self, artDisplay)

- מתודה שתוסיף מנוי לרשימת המנויים במוזיאון. המתודה תקבל אובייקט מסוג MuseumSubscriber ותוסיף אותו לרשימת המנויים במוזיאון.

add_subscriber(self, subscriber)

ג. הוסיפו למחלקה Museum מימוש למתודה:

change_preserving_date(self, name, new_date)

שתשנה תאריך שימור לאובייקט מסוג ArtDisplay בעל שם name שנמצא ברשימת המיצגים במוזיאון לתאריך שימור חדש new_date. הפונקציה תחזיר None. ניתן להניח שלכל מיצג במוזיאון יש שם ייחודי, ושהמיצג ששמו צוין נמצא במוזיאון. בהמשך לדוגמא הקודמת:

```
>>> museum.change_preserving_date("Mona", "24.6.2022")
```

```
>>> print (museum)
```

```
This museum contains the following displays:
```

```
lisa is a statue that was created in 30.5.83 and needs to be preserved in
24.6.2022
```

Mona is a paint that was created in 29.5.83 and needs to be preserved in 24.6.2022

ד. נרצה לדעת מהו השווי הכולל של מיצגי המוזיאון. הוסיפו למחלקה Museum מימוש למתודה `get_total_worth()` אשר תחזיר את סכום השווי של כל מיצגי המוזיאון.

ה. נרצה לממש אפשרות לבצע כניסה לאחד המנויים במוזיאון. הוסיפו למחלקה Museum מימוש למתודה `subscriber_entry(self, name)`. המתודה תקבל `(string) name`, ותעדכן כניסה למנוי (על ידי קריאה למתודה המתאימה באובייקט מסוג `MuseumSubscriber`) ששמו `.name`. ניתן להניח ששם המנוי קיים ברשימת מנויי המוזיאון פעם אחת בדיוק. ראו דוגמא בשאלה 4.

ו. נרצה לדעת מהו המיצג הכי אהוב במוזיאון. הוסיפו למחלקה Museum מימוש למתודה `find_loved_disp(self)` שתחזיר רשימה (list) של שמות המיצגים האהובים במוזיאון – כלומר, השמות של האובייקטים מסוג `ArtDisplay` שהופיעו הכי הרבה פעמים ברשימה `favorites` של כל מנוי במוזיאון. כל האובייקטים ששם מופיע ברשימה הינם בעלי אותו מספר הופעות ברשימות המנויים, וזהו המספר המקסימלי בין המיצגים השונים. ראו דוגמא בשאלה 4.

שאלה 4:

ממשו את הפונקציה `create_museum(filename)`. הפונקציה מקבלת שם של קובץ CSV המתאר את נתוני המיצגים והמנויים הכלולים במוזיאון ומחזירה אובייקט `Museum` (מלא בכל הנתונים שמופיעים בקובץ).

תיאור פורמט הקובץ:

- כל שורה בקובץ הקלט תכיל בתחילתה את שם המחלקה שיש להוסיף למוזיאון (מנוי או מיצג).
- סדר השדות יהיה על פי הסדר שהוגדר בחתימות המתודות של הבנאים.
- בשורות המציינות מנוי מוזיאון, שלושת השדות האחרונות בשורה מציינים את רשימת המיצגים האהובות על המנוי כרצף של מספרים המציינים את מספרי השורות בקובץ ה-CSV בו המיצגים הופיעו (ראו דוגמא). לכל מנוי בקובץ תהיינה בדיוק 3 מיצגים ברשימת המיצגים האהובים.
- ניתן להניח שבקובץ יופיעו תיאורי כל המיצגים לפני המנויים.
- ניתן להניח שכל הנתונים בקובץ תקינים, ללא חזרה של שמות, וללא מיצגים בעלי אותו שווי.

בסיום יצירת המוזיאון, יש להדפיס:

`This museum's worth is <museum worth>`

כך שבמקום `<museum worth>` יוצג סך השווי של המיצגים במוזיאון.

הבהרות:

1. הפונקציה גם מדפיסה את המצוין לעיל וגם מחזירה אובייקט מסוג Museum.

2. אין להניח שהקובץ קיים. במקרה של שגיאת IO, יש "לתפוס" אותה, ולהדפיס

Unable to load <filename> due to an IO Error

ולסיים את התכנית בצורה מסודרת. <filename> יוחלף בשם קובץ הקלט.

הקובץ museum.csv נמצא בין קבצי התרגיל:

	A	B	C	D	E	F
1	artDisplay	The Annunci	19.3.1474	painting	24.6.2023	70
2	artDisplay	Mona Lisa	18.7.1506	painting	24.6.2025	65
3	artDisplay	The Baptism	20.11.1472	painting	24.6.2022	90
4	artDisplay	David	18.7.1504	statue	13.7.2023	100
5	artDisplay	Moses	18.2.1501	painting	24.60.2020	30
6	subscriber	Dany	1	2	4	5
7	subscriber	Yael	5	3	5	2
8	subscriber	Jhon	12.1.2020	2	3	1

הסבר:

- השדה הראשון משמאל בכל שורה מתאר את סוג האובייקט שיש ליצור – מיצג או מנוי.
- עבור אובייקט מסוג מיצג, השדות החל מהשני והלאה יכילו את נתוני שם המיצג, תאריך היצירה, סוג המיצג, תאריך החידוש ושווי המיצג.
- עבור אובייקט מסוג מנוי, השדות החל מהשני והלאה יכילו את שם המנוי, סוג הכרטיס (ticket_type), ו-3 שדות בדיוק המציינים את האינדקסים של המיצגים האהובים ביותר על המנוי. האינדקס הזה מתאים לשורה של המיצג בקובץ הCSV. כך למשל, Dany אוהב את המיצגים Moses, David, Mona Lisa.

דוגמת הרצה עבור קובץ קלט זה:

```
>>> museum = create_museum("museum.csv")
```

```
This museum's worth is 355
```

```
>>> print (museum)
```

```
This museum contains the following displays:
```

```
Moses is a painting that was created in 18.2.1501 and needs to be preserved in 24.60.2020
```

```
Mona Lisa is a painting that was created in 18.7.1506 and needs to be preserved in 24.6.2025
```

```
The Annunciation is a painting that was created in 19.3.1474 and needs to be preserved in 24.6.2023
```


The Baptism of Christ is a painting that was created in 20.11.1472 and needs to be preserved in 24.6.2022

David is a statue that was created in 18.7.1504 and needs to be preserved in 13.7.2023

דוגמא לסעיף ו' בשאלה 3 :

```
>>> museum.subscriber_entry("Dany")
```

Welcome! 0 entries left

הסבר: לדני הוטען מנוי מסוג כניסה חד פעמית. לאחר שנכנס, אין לו יותר כניסות והוא צריך לשנות את המנוי.

דוגמא לסעיף ז' בשאלה 3 :

```
>>> print (museum.find_loved_disp())
```

['Mona Lisa']

הסבר: בקובץ ה CSV כל אחד מ-3 המנויים אוהב את מיצג מספר 2 בקובץ ה CSV וזהו המיצג האהוב ביותר. אם היה עוד מיצג שמופיע 3 פעמים ברשימת המיצגים האהובים של כל המנויים גם הוא היה נכנס לרשימה ומודפס.

שאלה 5:

א- ממשו קוד merge sort והעתיקו אותו ל-PDF (אין צורך לעלות אותו ב.py). למה האלגוריתם הזה עדיף מבחינת זמן ריצה? על מה מוותרים כדי לקבל זמן ריצה טוב?

להלן הקוד לחיפוש בינארי:

```
def binary_search_iterative(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid

        elif arr[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1 # Target not found
```

ב- הסבירו איך הקוד פועל.

ג- מה התלות של זמן ריצה של הקוד באורך הקלט לפי שיטת Worst Case Scenario?

בהצלחה !